

Core Concepts — From Text to Vectors

Part 03 — 🗣️ Natural Language Processing · 03-nlp/notes/01-text-to-vectors.md

1. Why language is hard for computers

- **Discrete and symbolic.** No natural distance metric: `cat` and `dog` are as far apart as `cat` and `carburettor`.
- **Ambiguous at every level.** "I saw her duck." Lexical, syntactic, and pragmatic ambiguity, all at once.
- **Compositional and unbounded.** Infinite valid sentences from a finite vocabulary.
- **Context-dependent.** "bank" changes meaning across a river and a loan.
- **Long-range dependencies.** The pronoun in sentence 12 refers to a noun in sentence 1.

Every technique below is an attempt to give language a geometry.

2. The pipeline

```
raw text
→ normalization      (unicode, casing, whitespace – lossy, decide deliberately)
→ tokenization        (text → integer IDs)
→ embedding lookup    (IDs → dense vectors)
→ contextualization    (vectors → context-aware vectors)    ← the transformer lives here
→ task head           (classification / generation / retrieval)
```

Everything in Modules 04–11 is this pipeline with a bigger middle.

3. A compressed history (know the shape of it)

Era	Idea	Limitation that killed it
1950s–80s	Rules and grammars	Language doesn't obey rules
1990s–2000s	n-gram language models , HMMs	Sparsity; no generalisation past seen n-grams
2001–2013	Bag-of-words, TF-IDF + linear models	No word order, no semantics
2013	word2vec / GloVe — static embeddings	One vector per word: "bank" is a compromise
2014–2017	seq2seq + RNN/LSTM + attention	Sequential → slow; long-range still leaky
2017	Transformer	(still here)
2018	ELMo, BERT , GPT-1 — transfer learning for NLP	—
2020–	Scale: GPT-3 onward	Module 04

word2vec's famous trick

`king - man + woman ≈ queen`. Training on the distributional hypothesis ("a word is known by the company it keeps") produces a space where analogies become vector arithmetic. Static embeddings are obsolete as a production choice, but the intuition — *meaning as direction in a vector space* — is exactly what powers Module 08's retrieval [08-rag].

4. Tokenization — the unglamorous source of half your bugs

A tokenizer converts text to integer IDs. The choice of granularity is a trade-off:

Granularity	Vocab size	Sequence length	Problem
Characters	~100	very long	model must learn spelling from scratch; expensive attention
Words	100k–1M+	short	out-of-vocabulary words; huge embedding table; no morphology
Subwords	30k–200k	moderate	the practical winner

Byte-Pair Encoding (BPE)

1. Start with a vocabulary of individual bytes/characters.
2. Count all adjacent pairs in the corpus. Merge the **most frequent pair** into a new token.
3. Repeat until you hit the target vocabulary size.

Frequent words become single tokens; rare words decompose into pieces. `tokenization` might become `token` + `ization`. Nothing is ever out-of-vocabulary because you can always fall back to bytes.

Variants: **WordPiece** (BERT — merges by likelihood rather than raw frequency), **SentencePiece/Unigram** (language-agnostic, treats the input as a raw byte stream, no pre-tokenization on whitespace — essential for Chinese, Japanese, Thai), **byte-level BPE** (GPT-2 onward — operates on bytes so any Unicode input is representable).

Tokenization explains a surprising amount of LLM weirdness

Observed behaviour	Tokenizer cause
Can't reliably count letters in "strawberry"	The model never sees letters — it sees ~2 tokens
Bad at arithmetic on long numbers	<code>1234567</code> splits inconsistently across numbers
Reversing a string is oddly hard	Character-level ops on subword units
Non-English costs 2–5× more tokens	Vocabularies are English-dominated
Trailing whitespace changes output quality	<code>hello</code> and <code>hello </code> are <i>different tokens</i>
"SolidGoldMagikarp"-style glitch tokens	Tokens in the vocab that were near-absent from training data
Code indentation matters to cost	Repeated spaces may be single tokens, or not

Practical rules: always count tokens with the *model's own* tokenizer, never `len(text)/4`. Never put a trailing space before a generation boundary. Budget ~ 1.3 tokens/word for English prose, and measure for anything else.

Build one yourself: `../code/bpe_tokenizer.py`. It takes 150 lines and permanently fixes your intuition.

5. Embeddings, static and contextual

- **Static** (word2vec, GloVe, fastText): one vector per word, fixed after training. `bank` gets one vector.
- **Contextual** (ELMo, BERT, GPT): the vector for a token depends on the whole sequence. `bank` in "river bank" and "bank loan" get different vectors. This is the single biggest quality jump in NLP history.

Sentence/document embeddings (what you'll actually use in RAG)

Averaging token vectors is a weak baseline. Better: models trained with a **contrastive objective** so that semantically similar sentences land close together — Sentence-BERT, E5, BGE, GTE, and the commercial embedding APIs. Key facts for Module 08 [08-rag]: - Cosine similarity is the usual metric; **normalize your vectors** and it becomes a dot product. - Asymmetric tasks (short query vs. long passage) often need **instruction prefixes** ("`query:` " / "`passage:` "). Getting these wrong silently degrades retrieval by a lot. - Dimensionality ($384 \rightarrow 3072$) trades quality for index size and latency. Matryoshka embeddings let you truncate.

6. Classical tasks (still shipped, still useful)

Task	What it does	Where it still matters
Tokenization/segmentation	split text	everywhere
POS tagging, parsing	grammatical structure	linguistics, some IE pipelines
Named Entity Recognition	find people/orgs/dates	PII redaction (Module 13), structured extraction
Coreference resolution	who does "she" refer to	document understanding, chunking (Module 08)
Sentiment / classification	label text	still the cheapest thing to fine-tune a small model for
Summarization	shorten faithfully	now mostly LLM territory
Semantic search	find by meaning	the retrieval half of RAG

A 2026 opinion worth holding: for a high-volume, narrow classification task, a fine-tuned 100M-parameter encoder is often $100\times$ cheaper and *more* accurate than prompting a frontier model. Don't reach for an LLM reflexively.

7. Evaluation in NLP

- **Perplexity** = $\exp(\text{average cross-entropy})$. "How surprised is the model by this text?" Lower is better. Comparable **only** between models with the same tokenizer and test set.
- **BLEU / ROUGE / METEOR** — n-gram overlap. Cheap, and weakly correlated with quality. Still reported.
- **BERTScore** — embedding similarity instead of exact overlap. Better, slower.
- **Exact match / F1** — extraction and QA.
- **Human evaluation and LLM-as-judge** — the current state of the art for generation. See Module 15 [15-ai-evals].

Perplexity measures *modelling*, not *usefulness*. A model can have excellent perplexity and be a terrible assistant.

8. Forward links

Idea here	Where it returns
BPE tokenization	Module 04 (cost, context), Module 11 (model comparison)
Contextual embeddings	Module 04 (the hidden states)
Sentence embeddings + cosine	Module 08 (vector retrieval)
Perplexity	Module 04 (scaling laws), Module 15 (evals)
NER	Module 13 (PII detection)
Encoder fine-tuning	Module 16 (cost-efficient production NLP)